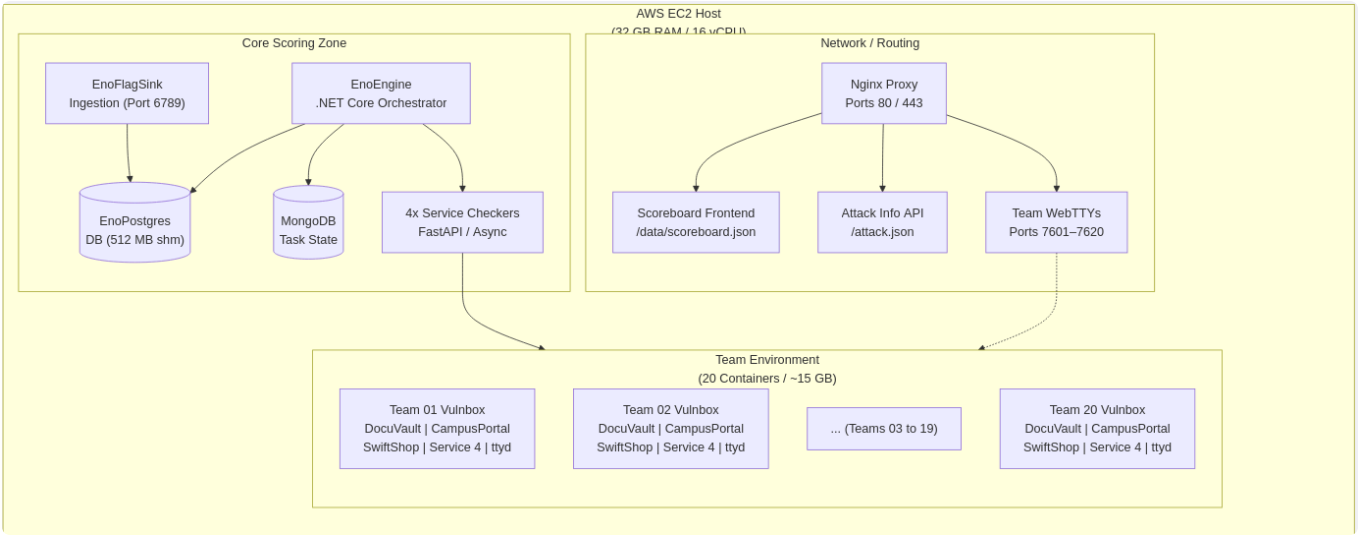
Executive Overview



1. System Architecture & Resource Allocation (32 GB RAM)

1.1 Memory Budgeting

Running 20 teams + EnoEngine on a single machine requires careful memory budgeting to avoid Linux OOM (Out-Of-Memory) killer invocations:

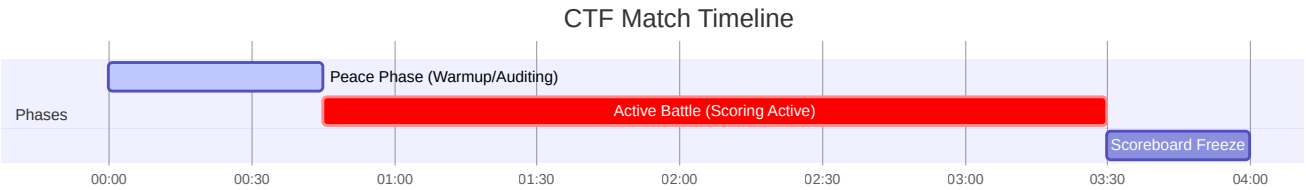
Component	Instances	RAM per Unit	Total Allocated RAM	Notes
Team Vulnboxes	20	768 MB (Capped at 1 GB)	**~15.0 GB**	All 4 services + WebTTY per team
EnoEngine Core	1	~250 MB	**0.25 GB**	C# .NET runtime orchestrator
EnoPostgres	1	~350 MB	**0.35 GB**	Postgres 15 with 512 MB shm
Service Checkers (4x)	4	~150 MB	**0.60 GB**	Python httpx async runners
MongoDB (ChainDB)	1	~400 MB	**0.40 GB**	State tracking for putflag/getflag
Scoreboard & Nginx	1	~100 MB	**0.10 GB**	Static frontend + reverse proxy

Component	Instances	RAM per Unit	Total Allocated RAM	Notes
Host OS & Docker Engine	1	~1.5 GB	**1.50 GB**	Kernel, systemd, dockerd, containerd
Buffer / Page Cache	—	—	**~13.8 GB**	Dynamic disk caching & buffer
Total Host Memory			**32.0 GB**	**Zero swap thrashing**

---

2. Event Game Design & Timeline (4-Hour Format)

2.1 Recommended Event Schedule

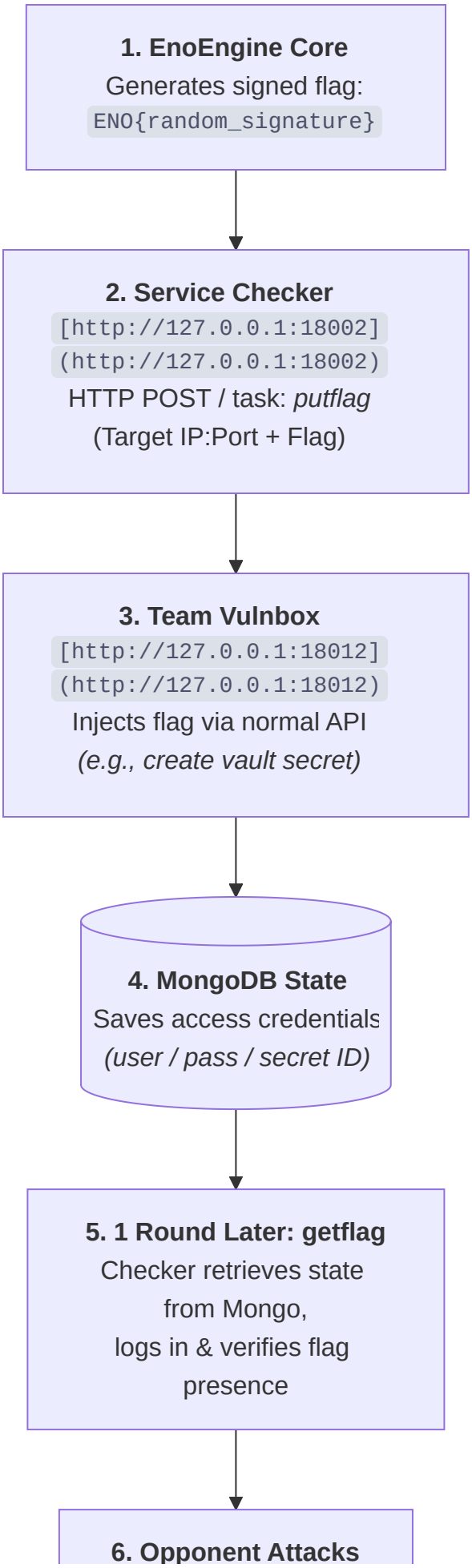


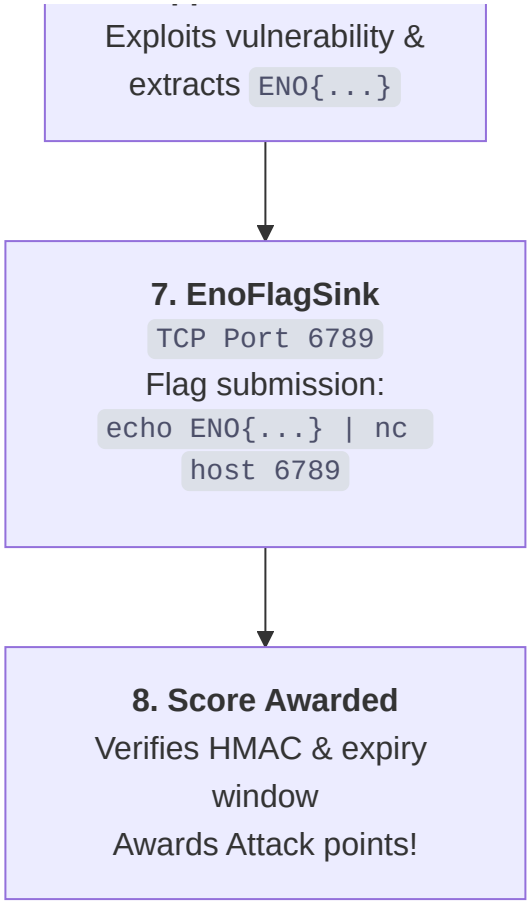
1. **Peace / Warmup Phase (T = 0:00 to 0:45 — 45 Minutes):**
   Let people read player guide and prepare defence script and understad services.
2. **Active Battle Phase (T = 0:45 to 3:30 — 165 Minutes):**
   90 Second rounds for 165 mins total 110 rounds. Scoreboard will continue to update giving updates about flags.
3. **Scoreboard Freeze Phase (T = 3:30 to 4:00 — Final 30 Minutes):**
   The score board stops to build suspense but the EnoFlagSink will still be planting flags.

---

3. How Checkers & Flag Injection Work

3.1 The Lifecycle of a Flag


1. EnoEngine Core
Generates signed flag:
ENO{random_signature}
2. Service Checker
[http://127.0.0.1:18002]
(http://127.0.0.1:18002)
HTTP POST / task: putflag
(Target IP:Port + Flag)
3. Team Vulnbox
[http://127.0.0.1:18012]
(http://127.0.0.1:18012)
Injects flag via normal API
(e.g., create vault secret)
4. MongoDB State
Saves access credentials
(user / pass / secret ID)
5. 1 Round Later: getflag
Checker retrieves state
from Mongo,
logs in & verifies flag
presence
6. Opponent Attacks



Port Allocation Scheme (20 Teams)

To eliminate port conflicts on the single host:

Resource	Base Port Formula	Example: Team 1	Example: Team 2	Example: Team 20
WebTTY Terminal	`7600 + TeamID`	`7601`	`7602`	`7620`
DocuVault	`10000 + (TeamID * 10) + 0`	`10010`	`10020`	`10200`
CampusPortal	`10000 + (TeamID * 10) + 2`	`10012`	`10022`	`10202`
SwiftShop	`10000 + (TeamID * 10) + 3`	`10013`	`10023`	`10203`
Service 4	`10000 + (TeamID * 10) + 4`	`10014`	`10024`	`10204`

Live Scoreboard Setup

EnoEngine writes the latest round results to `/home/ubuntu/EnoEngine/data/scoreboard.json` automatically at the end of every tick.

Nginx Static Scoreboard Server

Configure Nginx on the host to serve the Scoreboard and static assets:

`/etc/nginx/sites-available/scoreboard.conf`:

```
server {
    listen 80 default_server;
    server_name _;

    root /var/www/scoreboard;
    index index.html;

    # Expose live EnoEngine scoreboard data
    location /data/ {
        alias /home/ubuntu/EnoEngine/data/;
        autoindex off;
        add_header Cache-Control "no-store, no-cache, must-revalidate";
        add_header Access-Control-Allow-Origin *;
    }

    # Fallback to frontend
    location / {
        try_files $uri $uri/ /index.html;
    }
}
```
